

Lecture 04: Collections and Loops

Introduction to Programming with Kotlin

Agenda

1. Introduction to collections
2. repeat and while
3. for and the size property
4. Ranges
5. do-while and split

Why collections?

So far: one value per variable. Often you need **many** values of the same kind.

- A **list** stores an ordered sequence of elements
- Access by index: first element at index 0
- Today: read-only lists with `listOf`, and a peek at `mutableListOf`

Creating a list

```
1 fun main() {  
2     val nums = listOf(10, 20, 30)  
3     val names = listOf("Ada", "Grace", "Alan")  
4  
5     println(nums[0])           // 10  
6     println(names[2])         // Alan  
7     println(nums.size)        // 3  
8 }
```

`listOf(...)` creates a read-only list. Indexing uses `[i]`; length is `.size`.

Mutable lists (brief)

```
1 fun main() {  
2     val scores = mutableListOf(5, 7)  
3     scores.add(9)  
4     scores[0] = 6  
5     println(scores)           // [6, 7, 9]  
6 }
```

- `mutableListOf` — you can add/change elements
- The variable can still be `val` (the list object changes)
- Prefer read-only lists until mutation is needed

Indexing rules

- Valid indices: `0 ... size - 1`
- `list[list.size]` — out of bounds (crash)
- Empty list: `size == 0`; no valid index

Common mistake

Thinking the last index is `size`. It is `size - 1`.

Quiz 1 — Lists

Quiz

For `val xs = listOf("a", "b", "c")`, what is `xs[1]`?

1. "a"
2. "b"
3. "c"
4. 1

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer — Quiz 1

Answer

2. "b"

Indexing is zero-based: 0 → "a", 1 → "b", 2 → "c".

Repeating work

Loops run code multiple times.

- `repeat(n) { ... }` — do something exactly n times
- `while (condition) { ... }` — while the condition stays true

repeat

```
1 fun main() {  
2     repeat(3) {  
3         println("Hello")  
4     }  
5     repeat(5) { i ->  
6         println("Index $i")    // 0, 1, 2, 3, 4  
7     }  
8 }
```

The optional parameter is the iteration index starting at 0.

while

```
1 fun main() {  
2     var n = readln().toInt()  
3     while (n > 0) {  
4         println(n)  
5         n -= 1  
6     }  
7     println("Done")  
8 }
```

Check the condition **before** each iteration. If it starts false, the body never runs.

Avoid infinite loops

```
1 // Dangerous: n never changes
2 // while (n > 0) {
3 //     println(n)
4 // }
```

Ensure something in the loop moves toward making the condition false (counter update, new input, break, ...).

Try this

Read integers until the user enters 0, summing the rest.

Quiz 2 — while

Quiz

How many times does the body run?

```
var i = 0; while (i < 3) { i += 1 }
```

1. 0
2. 2
3. 3
4. Forever

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer — Quiz 2

Answer

3. 3 times

Iterations with `i` equal to 0, 1, then 2. After the third increment, `i == 3` and the loop stops.

`for` over a collection

The most common loop over lists:

- `for (item in list) { ... }`
- Visit each element in order
- No manual index required

Iterating elements

```
1 fun main() {  
2     val names = listOf("Ada", "Grace", "Alan")  
3     for (name in names) {  
4         println("Hello, $name")  
5     }  
6 }
```

Using `size` with indices

```
1 fun main() {  
2     val nums = listOf(4, 8, 15, 16)  
3     println("Count: ${nums.size}")  
4  
5     for (i in 0 until nums.size) {  
6         println("nums[$i] = ${nums[i]}")  
7     }  
8     // Or: for (i in nums.indices) { ... }  
9 }
```

0 until size is 0, 1, ..., size - 1 — safe indices.

Summing a list

```
1 fun main() {  
2     val xs = listOf(1, 2, 3, 4)  
3     var sum = 0  
4     for (x in xs) {  
5         sum += x  
6     }  
7     println(sum)    // 10  
8 }
```

Try this

Find the maximum element of a list of integers using a for loop.

Quiz 3 — for and size

Quiz

For a list with `size == 4`, which index range is safe?

1. `1..4`
2. `0..4`
3. `0 until 4`
4. `0..size`

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer — Quiz 3

Answer

3. `0 until 4`

Valid indices are 0, 1, 2, 3. `0 until 4` produces exactly those values; `0..4` would include 4 (out of bounds).

Ranges

A **range** is a sequence of values from a start to an end.

- `1..5` — inclusive: 1, 2, 3, 4, 5
- `1 until 5` — exclusive end: 1, 2, 3, 4
- `5 downTo 1` — descending
- `1..10 step 2` — 1, 3, 5, 7, 9

Looping over ranges

```
1 fun main() {  
2     for (i in 1..5) {  
3         print("$i ")  
4     }  
5     println()  
6     for (i in 5 downTo 1) {  
7         print("$i ")  
8     }  
9     println()  
10    for (i in 0 until 3) {  
11        print("$i ")  
12    }  
13 }
```

Output idea: 1 2 3 4 5 then 5 4 3 2 1 then 0 1 2.

in for membership

```
1 fun main() {  
2     val age = readln().toInt()  
3     if (age in 13..19) {  
4         println("Teen")  
5     }  
6     val letter = readln()[0]  
7     if (letter in 'a'..'z') {  
8         println("Lowercase letter")  
9     }  
10 }
```

Ranges work for integers and characters.

Quiz 4 — Ranges

Quiz

What values does `1 until 4` contain?

1. 1, 2, 3, 4
2. 1, 2, 3
3. 0, 1, 2, 3
4. 1, 4

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer — Quiz 4

Answer

2. 1, 2, 3

`until` excludes the end value. Inclusive form is `1..4`.

Like `while`, but the body runs **at least once**, then the condition is checked. Useful for “ask, then decide whether to ask again” menus and input loops.

do-while example

```
1 fun main() {  
2     var answer: String  
3     do {  
4         print("Type 'quit' to stop: ")  
5         answer = readln()  
6         println("You typed: $answer")  
7     } while (answer != "quit")  
8 }
```

Compare with `while`: here the prompt always appears at least once.

`String.split(...)` breaks text into a list of pieces.

- Common: split on spaces or commas
- Result type: `List<String>`
- Great for reading several values from one line

Reading several numbers on one line

```
1 fun main() {  
2     print("Enter numbers separated by spaces: ")  
3     val parts = readln().split(" ")  
4     var sum = 0  
5     for (p in parts) {  
6         if (p.isNotEmpty()) {  
7             sum += p.toInt()  
8         }  
9     }  
10    println("Sum = $sum")  
11 }
```

Common mistake

Forgetting that `split` returns strings — still need `toInt()` for arithmetic.

Split on commas

```
1 fun main() {  
2     val line = "Ada,Grace,Alan"  
3     val names = line.split(",")  
4     for (name in names) {  
5         println(name.trim())  
6     }  
7 }
```

`trim()` removes surrounding spaces after splitting.

Try this

Read a,b,c as three integers from one line and print the average.

Quiz

What is `"10 20 30".split(" ").size`?

1. 1
2. 2
3. 3
4. 10

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

3. 3

Splitting on spaces yields three strings: "10", "20", "30".

Summary

- Lists: `listOf`, index from 0, property size
- `repeat` / `while` / `for` / `do-while`
- Ranges: `..`, `until`, `downTo`, `step`
- `split` turns a string into a list of pieces

Looking ahead

Next: the `Char` type — single characters, codes, and how they relate to strings.